

Learning to Route Without Trusting the Model: Verified Rewards for Adaptive Retrieval Routers

Tarun Raja

[Affiliation]

[email]

Abstract—Adaptive retrieval-augmented generation (RAG) routers learn which retrieval strategy to use for each kind of question, and the usual reward is the generator’s self-reported confidence. We show that this makes the router learn the wrong quantity. Under a conditional-independence assumption, the expected self-reward of a strategy is an affine function of its correctness, with a slope that shrinks as the model grows overconfident. The reward-optimal and correctness-optimal strategies then coincide, but the Lai–Robbins regret lower bound, measured in correctness, grows as the inverse square of that slope, and learning stops entirely when the model is maximally overconfident. When the assumption fails, the two optima can differ, and regret is linear. We also show that replacing unobserved confidences with fallback constants biases every strategy’s estimate toward the constant, in proportion to its own failure rate. We then replace self-confidence with a *verified reward* built from signals checkable against something other than the model: claim-level grounding in the passages the chosen strategy retrieved, a sampled independent judge, and delayed human ratings. Unobserved signals are treated as missing rather than zero. In a controlled simulation using the deployed Thompson-sampling router, the verified reward cuts correctness regret by a factor of 1.9 to 7.2 across three scenarios. It raises the share of queries sent to the best strategy from 28–35% to 55–91%. The system is deployed in ContextWeave. We also release an offline calibration harness for evaluating the reward signals on public QA benchmarks.

Index Terms—retrieval-augmented generation, multi-armed bandits, Thompson sampling, reward design, calibration, LLM self-assessment

I. INTRODUCTION

Retrieval-augmented generation systems increasingly choose *how* to retrieve per question: pure vector search, graph-first expansion, keyword-boosted reranking, or a hybrid. Our earlier work [1] framed this as a contextual bandit over strategies. It learned a Beta posterior per (strategy, question type) pair, and each answer’s reward was the generator’s self-reported confidence. Thompson sampling [2], [3] fixed the exploration defect of an earlier greedy rule. But exploration fixes the *search*, not the *objective*. A strategy that retrieves the wrong passages, and gets a confidently wrong answer out of them, is reinforced exactly as much as one that retrieves the right ones.

This paper asks whether a router can learn reliably without trusting the model’s opinion of itself. Our contributions are:

- 1) A reward model that makes precise what a self-confidence-trained router learns (Section III). Under conditional independence the expected reward is affine

in correctness with slope s (Lemma 1). The Lai–Robbins bound in correctness units scales as $1/s^2$ (Theorem 1). Arm-dependent confounding or fallback substitution breaks the ordering itself (Propositions 1, 2).

- 2) A verified reward built from grounding, a sampled judge and human ratings, with explicit missingness (Section IV). Combining signals preserves the correctness ordering whenever each signal does (Proposition 3).
- 3) A simulation of the deployed router showing the regret reduction, and the reward’s sensitivity to overconfidence and to grounding noise (Section V).
- 4) An open offline harness that measures how far each signal sits from gold correctness on public QA data, with Brier score, ECE, AUROC and strategy-ranking agreement. The protocol is fixed; the model runs are pending (Section VI).

II. BACKGROUND AND RELATED WORK

Bandits. We use Thompson sampling with Beta posteriors [2], [3], [5]. For rewards in $[0, 1]$, Agrawal and Goyal [3] reduce to the Bernoulli case by drawing $B \sim \text{Bernoulli}(R)$. The deployed router instead applies the *fractional* update $\alpha \ += R$, $\beta \ += 1 - R$. Lai and Robbins [6] give the asymptotic lower bound on regret that any consistent policy must pay.

LLM self-assessment. Language models’ verbalized confidence is informative but miscalibrated, and is typically overconfident [7], [8]. Our result formalizes what that miscalibration costs a learner that uses it as a reward.

Grounding and judges. Faithfulness of an answer to its retrieved context is commonly measured with entailment models [9] or LLM judges [10], [11]. Judges exhibit position bias [11], which we address in a companion paper.

III. REWARD MODEL AND WHAT THE ROUTER LEARNS

Fix a question type. Strategy (arm) $a \in \{1, \dots, K\}$ produces a correct answer ($Y = 1$) with probability μ_a ; let $a^* = \arg \max_a \mu_a$ and $\Delta_a = \mu_{a^*} - \mu_a$. After each answer the router observes a reward $R \in [0, 1]$ or nothing ($\perp$). *Correctness regret* after T queries is $\text{Reg}_Y(T) = \sum_{t=1}^T \Delta_{a_t}$. It is measured in correctness, not in the reward the router sees, so a router that confidently learns the wrong thing shows up as regret.

Assumption 1 (Conditional independence). The reward depends on the arm only through correctness: $\mathbb{E}[R \mid Y, a, R \neq \perp] = \mathbb{E}[R \mid Y, R \neq \perp]$.

Lemma 1 (Affine reward). Under Assumption 1, let $c_y = \mathbb{E}[R \mid Y = y, R \neq \perp]$ and $s = c_1 - c_0$. Then the reward mean of arm a is $r_a = c_0 + s\mu_a$.

Proof. By the law of total expectation, $r_a = \mu_a c_1 + (1 - \mu_a)c_0 = c_0 + (c_1 - c_0)\mu_a$, using Assumption 1 to drop the conditioning on a in c_y . $\square$

For self-confidence, let an answer that is wrong still receive a high confidence ($\approx c_1$) with probability ρ , the *overconfidence*, and c'_0 otherwise. Then $c_0 = \rho c_1 + (1 - \rho)c'_0$, so $s_{\text{self}} = (1 - \rho)(c_1 - c'_0)$, which vanishes as $\rho \rightarrow 1$.

Theorem 1 (Regret scales as $1/s^2$). Consider Bernoulli-trick rewards: the router observes $B_t \sim \text{Bernoulli}(R_t)$, so arm a yields i.i.d. $\text{Bernoulli}(r_a)$ observations. Under Assumption 1 with $s > 0$:

- 1) the reward-optimal arm is a^* ;
- 2) for any policy that is consistent on the Bernoulli family,

$$\liminf_{T \rightarrow \infty} \frac{\mathbb{E}[\text{Reg}_Y(T)]}{\ln T} \geq \sum_{a \neq a^*} \frac{r_{a^*}(1 - r_{a^*})}{s^2 \Delta_a}; \quad (1)$$

- 3) if $s = 0$, every arm has the same reward law, and for any policy $\mathbb{E}[\text{Reg}_Y(T)] = \Theta(T)$ unless it happens to commit to a^* without information.

Proof. (1) By Lemma 1, r_a is strictly increasing in μ_a when $s > 0$.

(2) Lai and Robbins [6] show that for any policy consistent on the Bernoulli family, arm $a \neq a^*$ is pulled at least $(1 + o(1)) \ln T / \text{KL}(r_a, r_{a^*})$ times in expectation, where KL is the Bernoulli divergence. Each such pull costs Δ_a in correctness regret. By the χ^2 upper bound on KL, $\text{KL}(p, q) \leq (p - q)^2 / (q(1 - q))$. With $r_{a^*} - r_a = s\Delta_a$ this gives $\text{KL}(r_a, r_{a^*}) \leq s^2 \Delta_a^2 / (r_{a^*}(1 - r_{a^*}))$. Substituting into $\sum_a \Delta_a \mathbb{E}[N_a(T)]$ yields (1).

(3) If $s = 0$ then $r_a = c_0$ for all a , and the observations are identically distributed whatever arm is pulled. The policy’s choice sequence is therefore independent of which arm is a^* . Averaged over a uniformly random relabelling of arms, each arm is pulled T/K times in expectation, so $\mathbb{E}[\text{Reg}_Y(T)] = \frac{T}{K} \sum_a \Delta_a$. $\square$

Remark 1. Thompson sampling attains the Lai–Robbins constant on Bernoulli rewards [3], [4]. Theorem 1 therefore says that its correctness regret under self-reward is inflated by s_{self}^{-2} relative to a reward with slope 1. The fractional update has the same expected increment as the Bernoulli trick ($\mathbb{E}[B] = R$). We use the Bernoulli form for the proof; the regret simulation in `scripts/routing_regret_sim.py` runs both and finds them indistinguishable.

Proposition 1 (Confounding flips the optimum). If Assumption 1 fails, so that $r_a = c_0 + s\mu_a + \beta_a$ for arm-specific offsets

β_a , then whenever $\beta_b - \beta_{a^*} > s\Delta_b$ for some b , the reward-optimal arm is not a^* , and Thompson sampling incurs $\Omega(T)$ correctness regret.

Proof. $r_b - r_{a^*} = -s\Delta_b + \beta_b - \beta_{a^*} > 0$, so a^* is not reward-optimal. Thompson sampling is consistent in reward [3]: it pulls reward-suboptimal arms $O(\ln T)$ times. Hence a^* , being reward-suboptimal, is pulled $o(T)$ times, and each of the remaining $T - o(T)$ pulls costs at least $\min_{a \neq a^*} \Delta_a > 0$. $\square$

Such offsets arise when a strategy changes *how the model sounds* rather than how right it is. For example, a strategy that retrieves more text, correct or not, can yield a more confident synthesis.

Proposition 2 (Fallback constants bias toward the constant). Suppose that when a confidence is unobserved (omitted, unparseable, or a failed call) the pipeline substitutes a constant κ . Unobservation happens at arm-specific rate m_a , independently of R given the arm. Then the learned reward mean is $r'_a = (1 - m_a)r_a + m_a\kappa$, and the ordering of arms can reverse. Skipping unobserved rounds instead learns r_a exactly.

Proof. Total expectation over the observation indicator gives r'_a . Take two arms with $r_1 > r_2$, $m_1 > m_2$ and $\kappa < r_2$: then $r'_1 - r'_2 = (1 - m_1)r_1 - (1 - m_2)r_2 + (m_1 - m_2)\kappa$, which is negative for m_1 large enough. Under skipping, the retained observations are a sample of R given the arm, because missingness is independent of R given the arm (missing at random). Their mean is therefore r_a . $\square$

The deployed pipeline used fallbacks of 0.7, 0.5 and 0.0 for these three cases, so a strategy whose retrieval failed more often was pulled toward 0.0 regardless of its answers.

IV. VERIFIED REWARD DESIGN

Each signal is a value in $[0, 1]$ or $\perp$ (unobserved).

Grounding. The answer is split into sentence-level claims; fragments with fewer than three content tokens, and abstentions, are dropped. A claim is supported if its support against some retrieved passage reaches a threshold $\tau = 0.6$, and the signal is the fraction of claims supported. The default verifier is lexical: it takes the fraction of the claim’s content tokens present in the passage, and any number in the claim must appear verbatim, so “400 teams” against a passage saying 40 scores zero. The verifier is a parameter; an NLI cross-encoder can replace it. Grounding is $\perp$ when there are no passages, or when the answer contains no checkable claims.

Grounding is the signal that is *about the strategy*. Retrieval is the only thing the router chooses, and a strategy whose passages do not support the answer has failed at that job, whatever the generator thinks of the result.

Judge. An independent model (Claude Sonnet 4.6, grading Nova Pro’s answers) scores the answer against the same passages. It runs on a deterministic hash sample (5%) of query ids, so replays select the same queries. A failed or unparseable judge call yields $\perp$.

Human. Ratings arrive later through a feedback endpoint and are folded in with weight 2.

TABLE I

CORRECTNESS REGRET (MEAN $\pm$ S.D.) AND SHARE OF QUERIES SENT TO THE BEST STRATEGY; $T = 2000$, 40 SEEDS, $\rho = 0.7$, $\sigma_g = 0.2$.

Scenario	Reward	Regret	Best-arm share
flat_self	self	294.0 $\pm$ 25.9	0.323
	grounding	65.7 $\pm$ 17.8	0.795
	verified	62.9 $\pm$ 20.1	0.799
close	self	138.7 $\pm$ 12.2	0.277
	grounding	66.4 $\pm$ 21.3	0.588
	verified	71.4 $\pm$ 19.0	0.553
good_incumbent	self	383.5 $\pm$ 26.9	0.350
	grounding	52.6 $\pm$ 13.7	0.909
	verified	53.2 $\pm$ 10.9	0.909

Combination. Given the observed signals $\{(v_i, w_i)\}$, the reward is $R = \sum_i w_i v_i / \sum_i w_i$. If no signal was observed, the router is not updated.

Proposition 3 (Combination preserves order). *If each signal i satisfies Assumption 1 with slope $s_i > 0$, then R satisfies it too, with slope $\bar{s} = \sum_i w_i s_i / \sum_i w_i > 0$.*

Proof. Condition on the set of observed signals. Given Y , $\mathbb{E}[R | Y] = \sum_i w_i c_{i,Y} / \sum_i w_i$, which is independent of the arm. It is affine in Y with slope $\bar{s}$, and a positive combination of positive slopes is positive. $\square$

The self-confidence can still be included (verified+self), but by Proposition 3 it only dilutes the slope when $s_{\text{self}} < \bar{s}$. Every signal is recorded whether or not it is used, so each one’s agreement with human ratings can be measured from the same decision log.

V. SIMULATION

Setup. Four strategies with true correctness rates μ per scenario, and Thompson sampling with the deployed fractional update and priors, over $T = 2000$ queries and 40 seeds. Per query, correctness $Y \sim \text{Bernoulli}(\mu_a)$ is drawn, then three rewards:

- *self*: $\mathcal{N}(0.88, 0.06)$ if $Y = 1$, or with probability ρ if $Y = 0$; otherwise $\mathcal{N}(0.80, 0.12)$;
- *grounding*: $\mathcal{N}(0.8, \sigma_g)$ if $Y = 1$, else $\mathcal{N}(0.3, \sigma_g)$;
- *verified*: grounding, plus the judge on 5% of queries with weight 2.

All values are clipped to $[0, 1]$. Here $s_{\text{self}} \approx (1 - \rho) 0.08$ and $s_{\text{ground}} \approx 0.5$. The rates are assumptions chosen to reflect overconfident self-report [8]. The simulation therefore shows the *mechanism*; Section VI is what will measure it on real data.

Results. Table I shows that the verified reward cuts correctness regret by $1.9\times$ (close) to $7.2\times$ (good_incumbent), and raises the best-arm share from 28–35% to 55–91%. The largest gap is in *good_incumbent*, where self-reward pays the most for exploring: with a small slope, the router cannot tell that the incumbent is already best. At a 5% sample the judge adds little over grounding, as expected from its weight.

TABLE II

SENSITIVITY: FLAT_SELF REGRET VS. OVERCONFIDENCE ρ (20 SEEDS), AND CLOSE REGRET VS. GROUNDING NOISE σ_g (20 SEEDS).

ρ	self	grounding	verified
0.0	215.5	63.3	56.8
0.3	238.8	63.3	56.8
0.5	274.8	63.3	56.8
0.7	286.6	63.3	56.8
0.9	329.0	63.3	56.8
σ_g	self	grounding	verified
0.1	138.2	60.4	61.7
0.2	138.2	68.7	75.3
0.3	138.2	78.1	73.1
0.4	138.2	80.0	86.7

Table II matches Theorem 1 qualitatively: self-reward regret rises monotonically with ρ , while the verified rewards do not depend on ρ at all. Even at $\rho = 0$, self-reward regret is $3.4\times$ higher. That is because its slope (0.08) is small, not because it is wrong, which is exactly the $1/s^2$ effect. Grounding degrades gracefully as its own noise grows and stays below self-reward at every level tested. The finite-horizon ratios are smaller than the asymptotic $(s_{\text{ground}}/s_{\text{self}})^2 \approx 39$, as expected, since (1) is an asymptotic bound.

VI. OFFLINE CALIBRATION PROTOCOL

The simulation fixes the reward distributions by assumption. Whether lexical grounding is good enough on real answers is an empirical question, and we have fixed its protocol. For each record {question, strategy, retrieved passages, answer, self-confidence, gold answers} drawn from SQuAD 2.0 [13], Natural Questions or HotpotQA, correctness is token-F1 against gold [12], and an unanswerable question counts as correct iff the answer abstains. For each signal (self, lexical grounding, NLI grounding, judge) the harness reports: coverage (the fraction observed); Brier score; 10-bin ECE [14]; AUROC against correctness ($F1 \geq 0.5$); Spearman ρ with F1; and whether the signal ranks *strategies* in the same order as mean correctness. Ranking strategies correctly is the one property a router needs, and a poorly calibrated signal can still have it.

[Results pending: these runs require generating answers with each retrieval strategy on a local model. The harness (scripts/verified_reward_bench.py calibrate) is complete and tested; Table I’s mechanism predicts self-confidence will show high coverage, low AUROC and frequent failures of strategy-ranking agreement.]

VII. DEPLOYMENT NOTES

The verified reward runs in the query path of ContextWeave. Grounding costs no model call. The judge costs roughly \$0.0006 per query on average at the 5% sample. Each decision logs its selection propensity, self-confidence, grounding, judge score and reward, so a candidate policy can be evaluated offline before deployment (companion paper). Setting ROUTER_REWARD_SOURCE=self restores the previous behaviour for rollback.

VIII. LIMITATIONS

Lexical support cannot see negation or paraphrase: it will score “X is not Y” as supported by “X is Y”. The NLI verifier and the judge exist for that reason. Assumption 1 may fail for grounding as well, for example if a strategy retrieves long passages that support many claims by chance; the calibration protocol’s strategy-ranking check is designed to detect this. The simulation’s reward distributions are assumptions.

IX. CONCLUSION

A router trained on self-confidence learns the model’s opinion of itself. In the best case that costs a factor of s^{-2} in regret; in the worst it picks the wrong strategy forever. Rewards that can be checked against evidence, with missing treated as missing, restore both the objective and the learning rate in simulation. The real-data calibration protocol is ready to confirm or refute this on public benchmarks.

REFERENCES

- [1] T. Raja, [title of the adaptive RAG router paper], [venue, year].
- [2] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, pp. 285–294, 1933.
- [3] S. Agrawal and N. Goyal, “Analysis of Thompson sampling for the multi-armed bandit problem,” in *Proc. COLT*, 2012.
- [4] E. Kaufmann, N. Korda, and R. Munos, “Thompson sampling: An asymptotically optimal finite-time analysis,” in *Proc. ALT*, 2012.
- [5] D. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, “A tutorial on Thompson sampling,” *Found. Trends Mach. Learn.*, vol. 11, no. 1, 2018.
- [6] T. L. Lai and H. Robbins, “Asymptotically efficient adaptive allocation rules,” *Adv. Appl. Math.*, vol. 6, pp. 4–22, 1985.
- [7] S. Kadavath *et al.*, “Language models (mostly) know what they know,” arXiv:2207.05221, 2022.
- [8] M. Xiong *et al.*, “Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs,” in *Proc. ICLR*, 2024.
- [9] O. Honovich *et al.*, “TRUE: Re-evaluating factual consistency evaluation,” in *Proc. NAACL*, 2022.
- [10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” arXiv:2309.15217, 2023.
- [11] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS Datasets and Benchmarks*, 2023.
- [12] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016.
- [13] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proc. ACL*, 2018.
- [14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017.